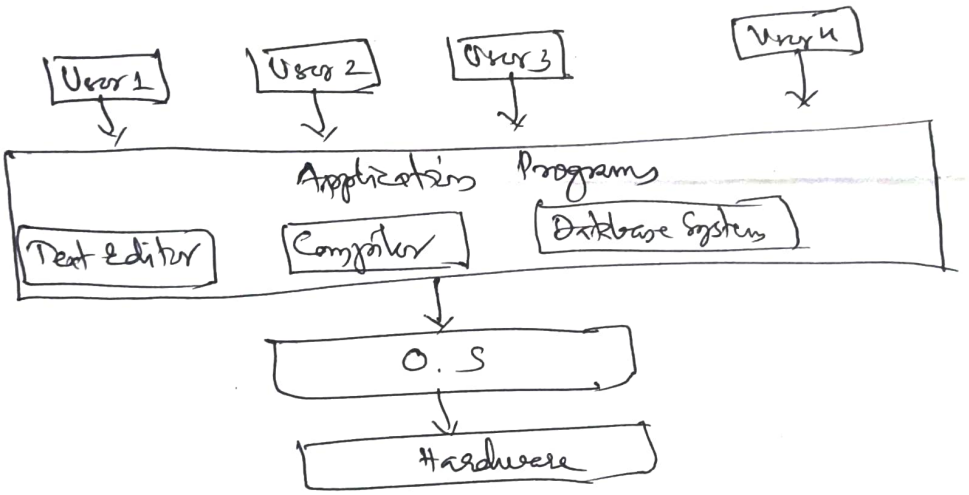
Operating System

is a software that manages and handle hardware and software resources of a computing device.

* Manages computer resources such as CPU, mby and files
* Acts as an interface between user and hardware
* Performs core functions like process, memory and file management
* Organizes system resources similar to different depts in a govt.

Eg Linux, Unix, Windows 11, ms-DOS, Android, Mac OS, iOS.

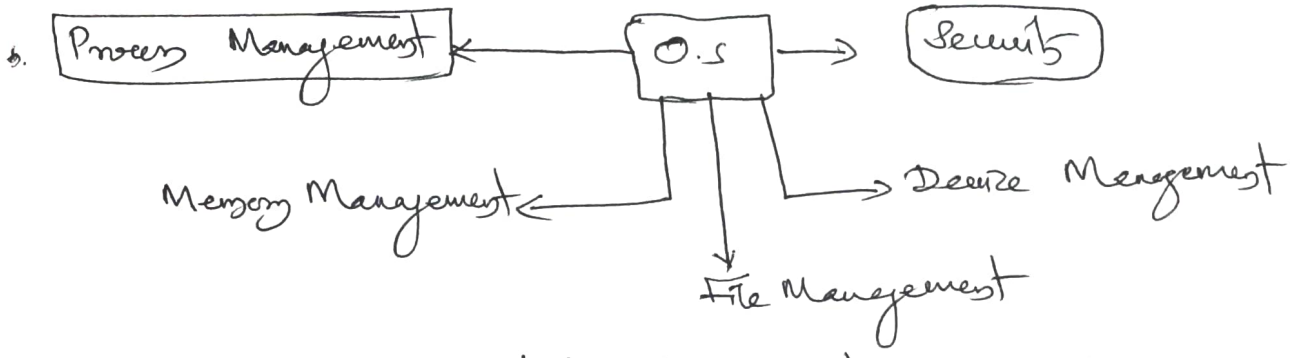


Components of an O.S

There are two basic components of an O.S.

* **Shell** - It is the outermost layer of the operating system and handles user interactions.
  * It interprets input for the O.S and handles the output from the O.S
* **Kernel** It is the core components of the O.S. The kernel is the primary interface between the operating system and hardware.

Functions of an O.S



The O.S. controls all computer resources, coordinates program execution, hides system complexity, ensures security, and improves resource utilization.

1. Process Management

* process management in operating system is about managing processes. A process is a running program.

Core functions

a) Scheduling :
Decides which process uses the CPU next

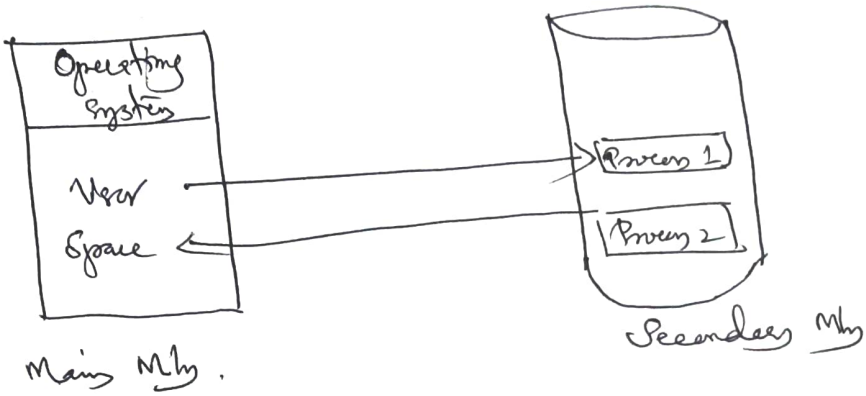
b) Synchronization
Ensures orderly execution using locks, semaphore, or monitors to prevent race conditions.

c) Deadlock handling
Detects & prevent situations where process wait forever for resources.

d) Inter-process Communication (IPC)
Allows processes to exchange data via shared mem or message passing

Memory Management

Memory management is an operating system controls how data is stored and organized in main memory and secondary storage, ensuring programs get the mly they need and freeing it when no longer used.



Key Activities

* **Allocation & Deallocation** :-
  Assigns and frees mly as needed.

* **Protection**
  Prevents processes from interfering with each other's mly.

* **Virtual Memory**
  Uses disk space as extra memory for very large programs.

* **Fragmentation Handling**
  Reduces wasted space through compaction

* **Disk Management**
  Handles file storage, free space, disk scheduling and backups

2

File System Management

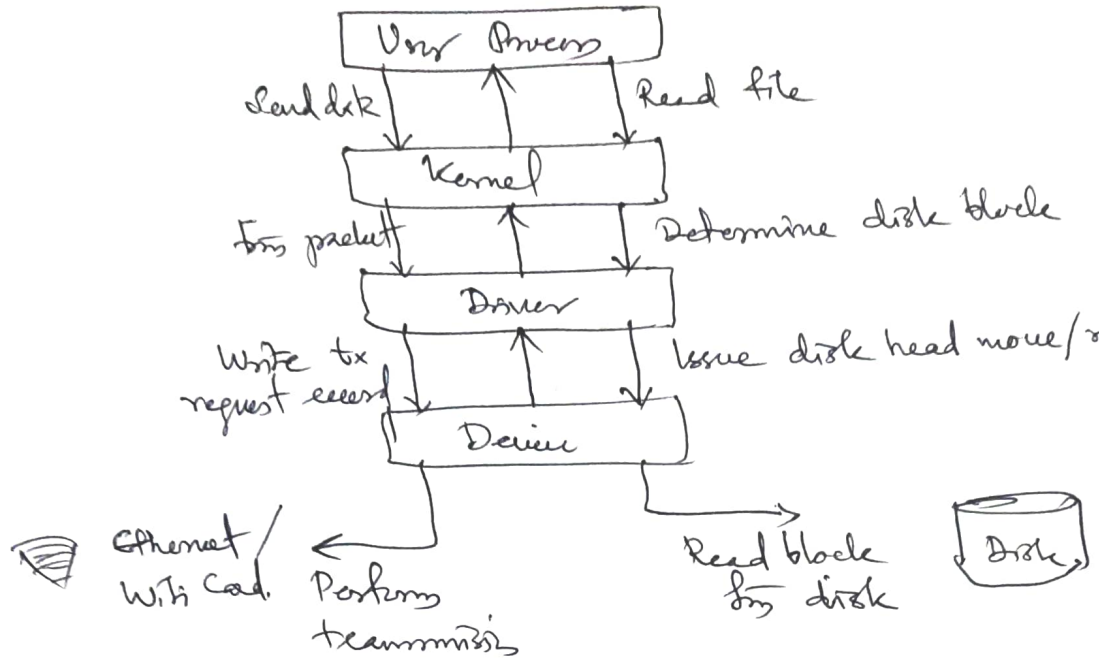
File management in an O.S organizes and controls how data is stored, named, and accessed on storage devices, making it easy for users to find and use their files.

File system management includes managing of

* **File Attributes** :-
  Names, type, size, and permissions.
* **File Types**
  Text, binary, and executable files
* **Operations**
  Create, read, write, and delete files.
* **Access Methods**
  Sequential, direct, and indexed access for faster retrieval.

Device Management (I/O System)

Device management in an O.S controls how the computer communicates with hardware devices like printers or disks, using drivers and techniques to ensure smooth and efficient operation.



Major Components in Device Management

- Device drivers
  - Interface between hardware & O.S
- Buffering & Caching
  - Temporarily store data to match device speeds and improve performance.
- Spooling :-

Spooling manages data waiting to be processed, particularly in devices like printers. The O.S places print jobs in a spool (a temporary storage area), allowing the CPU to continue other tasks while the printer works through the queue.

5. Protection & Security

Protection and security mechanisms in an O.S are designed to safeguard system resources from unauthorized access or misuse. These mechanisms control which processes or users can access specific resources (such as memory, files and CPU time). It also ensures that only authorized users can perform specific actions.

→ Access Control :
Limits user and process permissions.

→ Authentication :
Verifies users through credentials (UID/SID)

→ Resource Protection
prevents misuse of files, memory or devices.

→ Security
Guards against malware, denial of service attacks and data theft.

Process Synchronization

Process - Types

Independent process

⇓

Execution of one process does not affect the execution of others

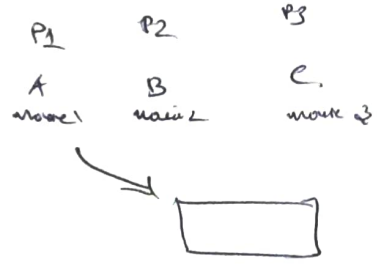
Cooperative Process

⇓

Execution of one process affects the execution of other process

Critical Section Problem

* It is a code segment, i.e, a shared variable
* Only one process can access the critical section at a time. During that time other processes are to be in wait state.




```
do
{
    Entry section
        Critical section
    Exit section
        Remainder section

} while (true);
```

Entry section — Controls the entry into critical section and gets lock on required resources.

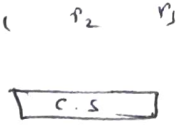
Exit section — Removes the lock on resources & others know that its C.S is over

Solution to C.S. Problem

A solution C.S. problem must satisfy the following three conditions.

1. **Mutual Exclusion**

If a process is executing its critical section, no other process is allowed to execute in C.S.

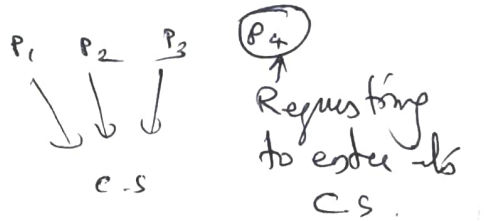


2. **Progress**

These processes wants to enter into C.S, that has to be allowed.

3. ~~Bounded~~ Waiting

~~After~~



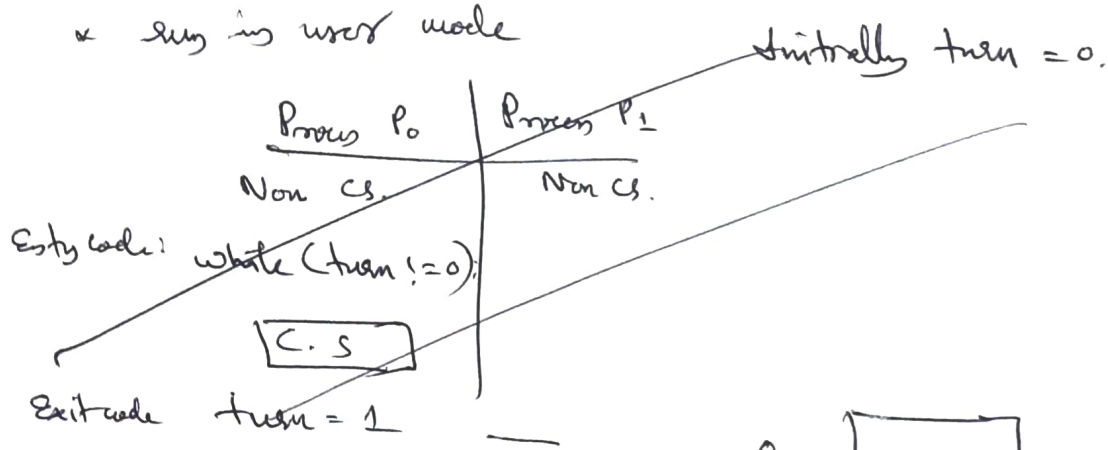
ii) If no process is executing in its C.S, and some processes wish to enter their C.S, then only those processes that are not executing in its remainder section can participate in the decision on which will enter its C.S next, and that selection cannot be postponed indefinitely.

3 **Bounded Waiting**

There exists a bound on the number of times that other processes are allowed to enter their C.S, after a process has made a request to enter its C.S, and before that request is granted.

Two process Solution

* Run in user mode





Algorithm 1

```
do
{
    while (turn != i);
        Critical section
    turn = j
        Remainder section
} while (1)
```

Spinlock or busy waiting

If the condition fails, it enter into its c.S.

In this algorithm, the processes share a common integer variable turn initialized to 0 (or 1).
If turn == i, then P_i is allowed to enter its c.S and execute and make turn = j.
If turn is j, then P_j is allowed to execute.
Here mutual exclusion ensures, but other requirements not satisfying.

Algorithm 2

* The problem with algo 1 is that it does not retain sufficient info about the state of each process; it remembers only which process is allowed to enter its C.S.
* Here we replace turn with the following array-

boolean flag[2]

The elements of the array is initialized to false
If flag[i] is true, this value indicates that P_i is ready to execute in C.S.

structure of P_i

```
do
{
Entry Section   flag[i] = true;
                while( flag[j]);

        critical Section

                flag[i] = false

        remainder section

} while(1).
```

In this algo; Process P_i first sets flag[i] to be true, signalling that it is ready to enter its C.S.

* Then P_i checks to verify that Process P_j is not also ready to enter its C.S.
* If P_j is ready, P_i would wait until P_j had indicated that it no longer needed to be in C.S. (i.e, until flag[j] was false)

- At that moment P_i would enter the C.S. On exiting the C.S, P_i would set flag[i] to be false, allowing the other process to enter its C.S.



P_i

```
do
{
    flag[i] = true;
    while (flag[j]);
        Critical Section
    flag[i] = false
        Remainder Section
}while(1)
```

```
do
{
    flag[j] = true;
    while( flag[i]);
        Critical section
    flag[j] = false
        Remainder section
}while(1)
```

Algorithm 3 (Peterson Solution)

Two process solution

Process P_i

```
while (true)
{
    --- Entry Section ---
    /* CS */
    -- remainder section --
}
```

Process share two variables :-

```
int turn;
boolean flag[2]
```

turn :- indicates whose turn to enter C.S

turn = i implies turn of Process P_i

flag :- used to indicate if a process is ready to enter C.S,

flag[i] = true implies Process P_i is ready.

Process P_i

```
while (true)
{
  flag[i] = true;
  turn = j;
  while (flag[j] && turn = j);
    /*   C.S   */
  flag[i] = false;
  /* remainder section */
}
```

Process P_j

```
while (true)
{
  flag[j] = true
  turn = i;
  while( flag[i] && turn =
  /*   C.S   */
  flag[j] = false
  /* Remainder section
```



Limitation

* Not guaranteed to work on modern architecture
  - To improve performance, processors and/or compilers may reorder operations that have no dependencies.
* Provides good algorithmic descriptions of solving C.S problems
* Illustrates some complexities involved in designing S/W that addresses requirements of mutual exclusion, progress, bounded waiting.

IPC

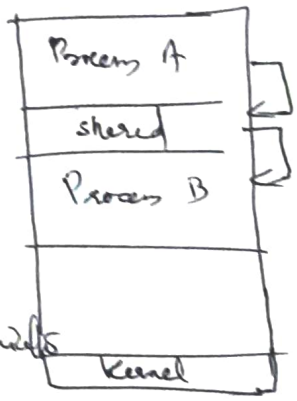
* There are two types of processes
  - Independent
  - Co-operating

Co-operating system processes require some type of IPC. This is allowed by two models.

1. Shared Memory System
2. Message Passing

① Shared Memory

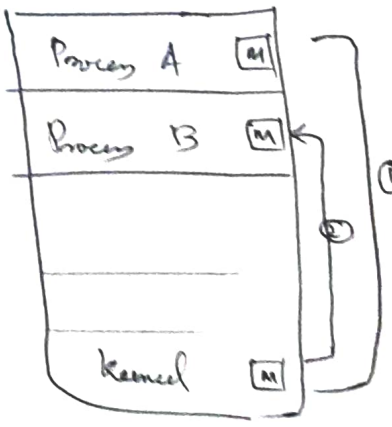
A region of shared memory is created within the address space of a process, which needs to communicate



* other process that needs to communicate uses this shared mem.
* Generally a few messages must be passed to and fro between the co-operating processes first in order to set up and coordinate the shared mem access.
* The process should take care that the two processes will not write the data to the shared mem at the same time.

② Message Passing

Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space and is particularly useful in a distributed environment, where the communicating processes



are reside on different computers connected by a network.

* A message passing facility provides at least two operations:

send (message) & receive (message)

Messages sent by a process can be of either fixed size or variable one.

* If process P & Q want to communicate, then must send messages to and receive messages from each other, a communication link must exist between them.

* The link can be implemented in a variety of ways.

1. Direct or indirect communication
2. Synchronous or asynchronous "
3. Automatic or explicit buffering.

* Under direct communication, each process that wants to communicate must explicitly ~~the~~ name the recipient or sender of the communication.

* Here send() & receive() primitives are defined as

- send (P, message)
  - send a message to process P.
- receive (Q, message)
  - Receive a message from process Q.

In indirect, the messages are sent to and received from mailboxes or ports.

* Each mailbox has a unique identification
* Eg POSIX message queues uses an integer value to identify a mailbox.
* Two processes can communicate only if the processes have a shared mailbox

send (A, message)
- Send a message to mailbox A

receive (A, message)
- Receive a message from mailbox A

Synchronization

Message passing may be either blocking or non blocking also known as synchronous & asynchronous.

- blocking send.
The sending process is blocked until the message is received by the receiving process or by the mailbox
- Non blocking send.
The sending process sends the message and resumes its operation

- Blocking receive
  - The receiver blocks until a message is available
- Non blocking receive
  - The receiver retrieves either a valid message or null.

Buffering

The messages are stored in temporary queues. Basically such queues can be implemented in three ways.

- Zero capacity

The queue has a maximum length of zero. Thus, the link cannot have any messages waiting in it. The sender must block until the recipient receives the message.

- Bounded Capacity

The queue has finite length n, so that at most n messages can reside in it. If the queue is not full, when a new message is sent, the message is placed in the queue, and the sender can continue execution without waiting. If the link is full, the sender must block until space available in the queue

- Unbounded Capacity

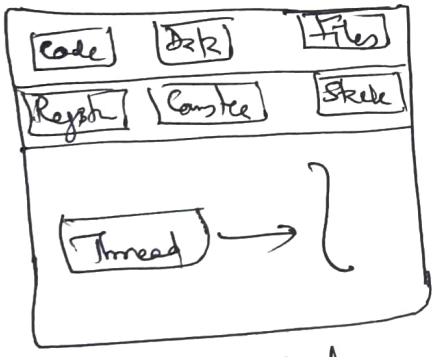
The queue length is potentially infinite; any no. of messages can wait in it. The sender never blocks.

Threads

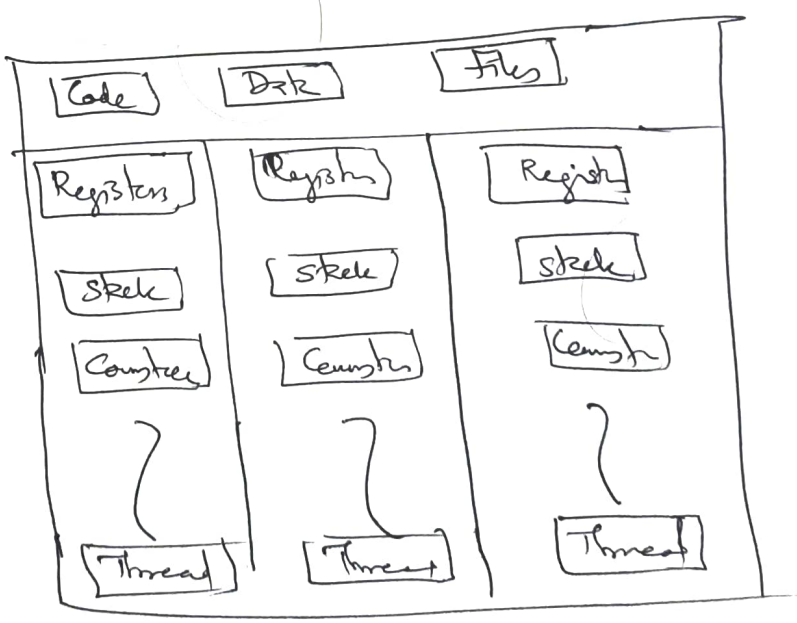
A thread is a light weight process because it is smaller and faster.

→ It allows multiple tasks to run simultaneously improving program efficiency.

* In single core systems, it creates an illusion of parallelism; In multicore systems, threads can execute truly in parallel across different cores.
* Each thread has its own program counter, register set, and stack space.
* Threads share code, data and operating system resources within the same process.



Single threaded process.



Need of threads

Threads are needed in modern operating system and applications because

* Improves performance
  - Threads allow multiple tasks to run at the same time (parallel or interleaved), making programs execute faster.

* Increase Responsiveness
- If one thread is busy, another can handle user actions, keeping the application responsive.

* Enable Concurrency:
Multiple operations like saving files, processing data, and handling user input can happen simultaneously.

* Better CPU utilization
On multicore systems, threads can run on different cores, improving overall system performance.

* Efficient Resource Sharing
Threads share the same memory and resources within a process, making communication faster and reducing overhead.

Components of Threads

1. Stack Space:
Store local variable, function calls, and return addresses specific to the thread.

2. Register Set:
Hold temporary data and intermediate results for the thread's execution.

3. Program Counter:
Tracks the current instruction being executed by the thread.

Types of threads

1. User level thread
2. Kernel level thread.

User level thread

* All thread management is done by the application.
* The kernel is not aware of the existence of threads.
* Thread creation and scheduling are done in user space.
* User level threads are fast to create and manage.
* In single threaded kernel, blocking system call from user level thread will block the entire process even if other threads are ready to run.

Kernel level thread

* Supported directly by the operating system kernel.
* Creation, scheduling, management are done by kernel in Kernel space.
* They are slower to create and manage.
* In multiprocessor kernel can schedule threads on different processors.

Inter process Communication (IPC)

Inter processes Communication (IPC) is a mechanism that allows processes to communicate and share data with each other while they are running.

- Since each process has its own memory space, IPC provides controlled methods for exchanging information and coordinating actions.
- It helps work processes work together efficiently and safely in an O.S.
- It helps processes synchronize their actions, share information and avoid conflicts while accessing shared resources.

There are two methods.

1. Shared memory Model
2. Message passing model

Shared memory

Communications between processes using shared memory requires processes to share a memory segment, the implementation of which is handled by the programmer. Process can use shared memory for extracting information as a record from another process as well as for delivering any specific information to other process.

Process

* A process is a program in execution
* A process is more than the program code, sometimes known as text section.
* It includes the current activity as represented by the value of the program counter and the content of the processor registers
* A process generally includes the process stack, which contains temporary data (such as method parameters, return addresses, and local variables), and a data section containing global variables.

Process address space

Each process has/gets its own virtual memory address space that consists of

1. stack space - used for : functions & system calls
2. Data space - variables (static & dynamic)
3. Text. - program code



Process state

The state of a process is defined in part by the current activity of that process. Each process may be in one of the following states.

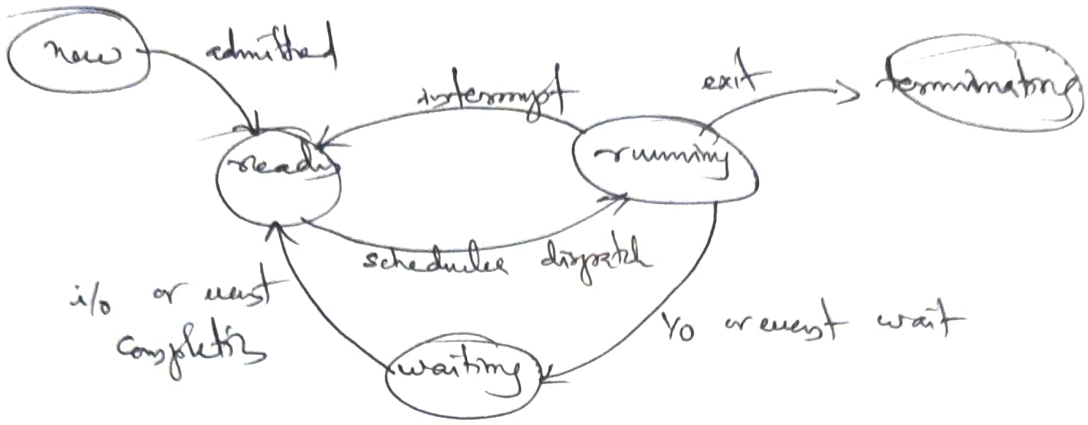
* New: The process is being created
* Running: Instructions are being executed
* Waiting: The process is waiting for some event to occur. (ie, i/o completion, or receiption of a signal)

* Ready:

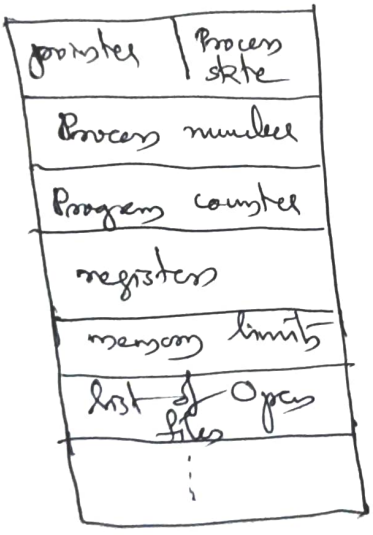
The process is waiting to be assigned to a processor

* Terminated

The process has finished execution.



Process Control Block (PCB)



Process state - new, ready, running, waiting, terminated

Program counter - indicates the address of the next instruction to be executed.

CPU registers - It includes accumulator, index register, stack pointer, general purpose register + condition code information.

CPU scheduling information: This information includes a process priority, pointers to scheduling queues, and any other scheduling information

Memory management information

The values of the base and limit registers, the page tables or the segment tables, depending on the memory system used by the O.S

Accounting information

It includes CPU time used, time limits, execution/account numbers (identifiers associated with the process for tracking system usage), clock time/timestamps, etc

I/O status information

It tracks all input and output operations linked to the process

Operations On Process

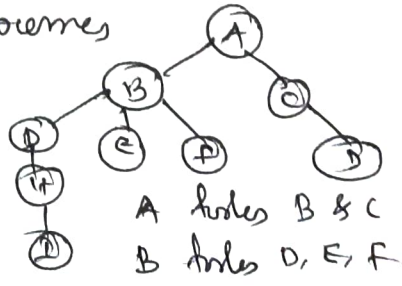
1. Process Creation
2. Process Termination

Process Creation

During the process execution, a process may create several new processes.

* The creating process is called a parent process and the new processes are called children of that process

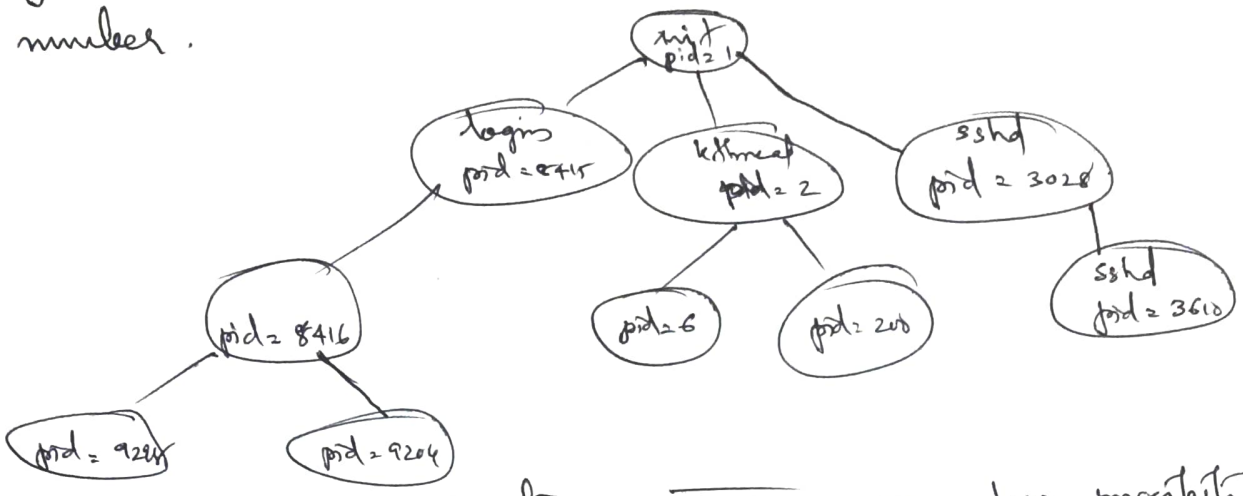
* Each of these new processes may in turn create other processes, forming a tree of processes



*

12

Operating systems identify processes with a unique process identifier (pid) which is typically an integer number.



When a process creates a new process, two possibilities for execution exist:

1. The parent continues to execute concurrently with its children.
2. The parent waits until some or all of its children have terminated.

→ There are two address space possibilities for the new process.

1. The child process is a duplicate of the parent process (it has the same program and data as the parent)
2. The child process has a new program loaded into it.

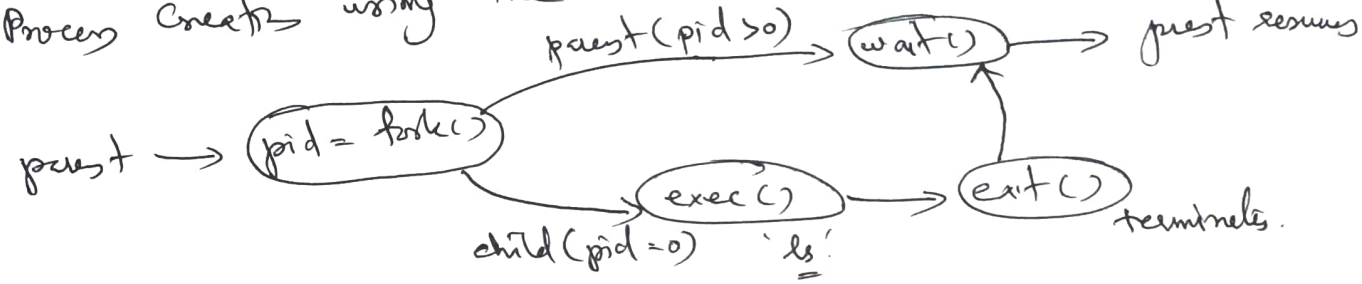
Resource allocation to child processes

* When a process creates a child process, that child process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task
* A child process may be able to obtain its resources directly from the operating system, or

it may be constrained to a subset of the resources of the parent process.

* The parent may have to partition its resources among its children, or it may be able to share some resources (such as memory or files) among several of its children.

Process Creation using the fork() system call



Process Termination

A process terminates when it finishes executing and asks the operating system to delete it by using the exit() system call.

* At that point, the process may return a status value (typically as integer) to its parent process (via wait() system call)

* All the resources of the process - including physical and virtual mem, open files, and I/O buffers are deallocated by the operating system.

⇒ A parent may terminate the execution of one of its children for a variety of reasons such as

- The child has exceeded its usage of some of the resources that it has been allocated.
- The task assigned to the child is no longer required.

- The parent is exiting, and the operating system does not allow a child to continue if its parent terminates.

- If a process terminates (either normally or abnormally) then all its children must also be terminated.
  - This phenomenon referred to as cascading termination is normally initiated by the operating system.

Scheduler

A scheduler in an O.S is a core system component that manages multitasking by deciding which processes gain access to the CPU.

→ It transitions processes between various scheduling queues (i.e. ready & waiting queue) to maximize processor utilization and minimize task completion time.

Types of schedulers

1. Long-Term (Job) Scheduler

Selects programs from secondary mem (hard drive) and loads them into main mem RAM. It controls the degree of multiprogramming and balances CPU-bound vs. I/O bound processes.

2. Short term (CPU) scheduler

Extremely fast and frequent, this scheduler selects the next ready process in RAM to execute on the CPU.

3. Medium-Term Scheduler

Manages memory by temporarily swapping blocked or idle processes out to the disk (to free up RAM) and swapping them back in when they resume.
